

Introductory

Two Knights

```
#include

using namespace std;
typedef long long ll;

int N;

int main(){
    scanf("%d", &N);
    for(int k = 1; k <= N; k++){
        ll cnt = 1LL + (k-1)*(k-2)/2;
        cnt = cnt * (k-1) * (k+4);
        printf("%lld\n", cnt);
    }
}
```

Trailing Zeros

```
#include

using namespace std;

int N, ans;

int main(){
    scanf("%d", &N);
    while(N > 0){
        N /= 5;
        ans += N;
    }
    printf("%d\n", ans);
}
```

Gray Code

```
#include

using namespace std;
const int maxN = 16;

int N;
bool b[maxN+1];
```

```
void print(){
    for(int i = N; i > 0; i--){
        printf("%d", b[i]);
        printf("\n");
    }

    int main(){
        scanf("%d", &N);

        print();
        for(int i = 1; i < (1<
```

Tower of Hanoi

```
#include

using namespace std;

int N;

void move(int from, int to, int depth){
    if(depth == 1){
        printf("%d %d\n", from, to);
        return;
    }

    int other = 6 - from - to;
    move(from, other, depth-1);
    printf("%d %d\n", from, to);
    move(other, to, depth-1);
}

int main(){
    scanf("%d", &N);
    printf("%d\n", (1<
```

Creating Strings

```
#include

using namespace std;

int N;
char S[9];
set perms;

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    sort(S, S+N);
```

```
perms.insert(S);
while(next_permutation(S, S+N))
    perms.insert(string(S, S+N));

cout << perms.size() << '\n';
for(string perm : perms)
    cout << perm << '\n';
}
```

Apple Division

```
#include

using namespace std;
typedef long long ll;
const ll INF = 0x3f3f3f3f3f3f3f;

int N, p[20];
ll a, b, best;

int main() {
    scanf("%d", &N);
    for (int i = 0; i < N; i++)
        scanf("%d", &p[i]);

    best = INF;
    for (int i = 0; i < (1 << N); i++) {
        a = 0;
        b = 0;
        for (int j = 0; j < N; j++) {
            if (i & (1 << j))
                a += p[j];
            else
                b += p[j];
        }
        best = min(best, abs(a - b));
    }
    printf("%lld\n", best);
}
```

Chessboard and Queens

```
#include

using namespace std;
const int N = 8;

char c;
int ans;
bool b[N][N], input[N][N];

bool place(int x, int y){
```

```

bool yes = true;
for(int i = 0; i < N; i++)
    if(b[x][i] || b[i][y])
        yes = false;
for(int i = 0; x-i >= 0 && y-i >= 0; i++)
    if(b[x-i][y-i])
        yes = false;
for(int i = 0; x-i >= 0 && y+i < N; i++)
    if(b[x-i][y+i])
        yes = false;
return yes;
}

bool check(){
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            if(b[i][j] && input[i][j])
                return false;
    return true;
}

void dfs(int i){
    if(i == N){
        if(check())
            ans++;
        return;
    }
    for(int j = 0; j < N; j++){
        if(place(i, j)){
            b[i][j] = true;
            dfs(i+1);
            b[i][j] = false;
        }
    }
}

int main(){
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            scanf(" %c", &c);
            input[i][j] = (c == '*');
        }
    }
    dfs(0);
    printf("%d\n", ans);
}

```

Raab Game I

```

#include

using namespace std;

int T;

int main(){
    scanf("%d", &T);
    for(int t = 0, n, a, b; t < T; t++){
        scanf("%d %d %d\n", &n, &a, &b);

        // Degenerate case: all ties
    }
}

```

```

if(a == 0 && b == 0){
    printf("YES\n");
    for(int i = 0; i < 2; i++)
        for(int j = 1; j <= n; j++)
            printf("%d%c", j, (" \n")[j==n]);
    continue;
}

// Otherwise, each player must win at least once
// and the total wins cannot exceed n
if(a + b > n || a == 0 || b == 0){
    printf("NO\n");
    continue;
}

// General case: trade wins then handle ties
printf("YES\n");
int ties = n - a - b;
vector xs(n+1);

// Start with both players playing 1, 2, 3, ...
// Cyclic shift a player by k to the right
// in order to give them k wins
// E.g. n = 5, shift b by k = 2
// as = 1 2 3 4 5 (3 wins)
// bs = 4 5 1 2 3 (2 wins)
for(int i = 1; i <= a+b; i++){
    int pos = (i + b - 1) % (a + b) + 1;
    xs[pos] = i;
}
for(int i = a+b+1; i <= n; i++)
    xs[i] = i;

// Print out solution
for(int i = 1; i <= n; i++)
    printf("%d%c", i, (" \n")[i==n]);
for(int i = 1; i <= n; i++)
    printf("%d%c", xs[i], (" \n")[i==n]);
}
}

```

Mex Grid Construction

```

#include

using namespace std;

int N;

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            printf("%d%c", i ^ j, (" \n")[j==N-1]);
}

```

Knight Moves Grid

```
#include

using namespace std;
typedef pair pii;
const int maxN = 1e3+5;

const int M = 8;
const int h[M] = {2, 2, 1, -1, -2, -2, -1, 1};
const int v[M] = {-1, 1, 2, 2, 1, -1, -2, -2};

int N, dist[maxN][maxN];
bool vis[maxN][maxN];
queue Q;

bool in_bounds(int x, int y){
    return 0 <= x && x < N && 0 <= y && y < N;
}

int main(){
    scanf("%d", &N);

    Q.push({0, 0});
    vis[0][0] = true;
    while(!Q.empty()){
        pii P = Q.front();
        Q.pop();

        int x = P.first, y = P.second;
        for(int i = 0; i < M; i++){
            int dx = x + h[i], dy = y + v[i];
            if(in_bounds(dx, dy) && !vis[dx][dy]){
                dist[dx][dy] = dist[x][y] + 1;
                vis[dx][dy] = true;
                Q.push({dx, dy});
            }
        }
    }

    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            printf("%d%c", dist[i][j], (" \n")[j==N-1]);
}
```

Grid Coloring I

```
#include

using namespace std;

int N, M;

const char cs[2][2] = {{ 'A', 'B' }, { 'C', 'D' }};

int main(){
    scanf("%d %d", &N, &M);
    for(int i = 0; i < N; i++){
```

```
    for(int j = 0; j < M; j++){
        char c;
        scanf(" %c", &c);

        bool odd = (i+j)&1;
        bool small = c < 'C';
        printf("%c", cs[small][odd]);
    }
    printf("\n");
}
```

Digit Queries

```
#include

using namespace std;
typedef long long ll;

int Q;
ll K;

ll pow10(int x){
    ll res = 1;
    for(int i = 0; i < x; i++)
        res *= 10;
    return res;
}

int solve(ll N){
    if(N < 9)    return (int) N+1;

    int len = 1;
    while(9 * pow10(len-1) * len < N){
        N -= 9 * pow10(len-1) * len;
        len++;
    }

    string S = to_string(pow10(len-1) + N/len);
    return (int) (S[N%len] - '0');
}

int main(){
    scanf("%d", &Q);
    for(int q = 0; q < Q; q++){
        scanf("%lld", &K);
        printf("%d\n", solve(K-1));
    }
}
```

String Reorder

```

#include

using namespace std;
const int maxN = 1e6+5;

int freq[26];
char S[maxN], ans[maxN];

bool possible(int current){
    int mode = 0, total = 0;
    for(int c = 0; c < 26; c++){
        if(freq[mode] < freq[c])
            mode = c;
        total += freq[c];
    }
    return (freq[mode] <= (total+1)/2) && (freq[current] <= total/2);
}

int main(){
    scanf("%s", S);
    const int N = (int) strlen(S);
    for(int i = 0; i < N; i++)
        freq[(int) (S[i] - 'A')]++;

    int lastchar = -1;
    for(int i = 0; i < N; i++){
        bool filled = false;
        for(int c = 0; c < 26 && !filled; c++){
            if(freq[c] > 0 && c != lastchar){
                freq[c]--;
                if(possible(c)){
                    ans[i] = (char) (c + 'A');
                    lastchar = c;
                    filled = true;
                } else freq[c]++;
            }
        }

        if(!filled){
            printf("-1\n");
            return 0;
        }
    }

    for(int i = 0; i < N; i++)
        printf("%c", ans[i]);
    printf("\n");
}

```

Grid Path Description

```

#include

using namespace std;
const int N = 7;

int ans;
char c[N*N+1];
bool vis[N+1][N+1];

```

```
bool inbounds(int x, int y){
    return 1 <= x && x <= N && 1 <= y && y <= N;
}

void dfs(int x, int y, int i){
    if(i == N*N-1 || (x == N && y == 1)){
        ans += (i == N*N-1 && (x == N && y == 1));
        return;
    }

    if((!inbounds(x+1, y) || vis[x+1][y]) && (!inbounds(x-1, y) || vis[x-1][y]))
        if(inbounds(x, y-1) && !vis[x][y-1] && inbounds(x, y+1) && !vis[x][y+1])
            return;
    if((!inbounds(x, y+1) || vis[x][y+1]) && (!inbounds(x, y-1) || vis[x][y-1]))
        if(inbounds(x+1, y) && !vis[x+1][y] && inbounds(x-1, y) && !vis[x-1][y])
            return;

    vis[x][y] = true;
    if(c[i] == 'D' || c[i] == '?')
        if(inbounds(x+1, y) && !vis[x+1][y])
            dfs(x+1, y, i+1);
    if(c[i] == 'U' || c[i] == '?')
        if(inbounds(x-1, y) && !vis[x-1][y])
            dfs(x-1, y, i+1);
    if(c[i] == 'R' || c[i] == '?')
        if(inbounds(x, y+1) && !vis[x][y+1])
            dfs(x, y+1, i+1);
    if(c[i] == 'L' || c[i] == '?')
        if(inbounds(x, y-1) && !vis[x][y-1])
            dfs(x, y-1, i+1);
    vis[x][y] = false;
}

int main(){
    scanf("%s", c);
    dfs(1, 1, 0);
    printf("%d\n", ans);
}
```